沪深 300 指数增强与截面选股挑战赛
提交指南

CSI 300 Index Enhancement & Cross-Sectional Stock Selection Challenge

重要：本指南是提交格式的唯一标准。

提交前请对照第七节的自检清单逐条核对。

配套的 `submission_template/` 是一份可直接运行的合规样例，建议参赛者按照其结构修改。

Submission Guide

1. 总览

参赛者需要提交一个**推理包**：

- 一个 `inference.py` 推理入口文件；
- 一个保存模型训练产物的 `model/` 目录；
- 一个能够复现模型的 `train.py`；
- 一个声明运行依赖的 `requirements.txt`。

平台在评测过程中会加载参赛者的 `inference.py`，随后逐个信号日传入截至当日能够获得的数据，并收集模型输出的 `score`。

评测系统根据模型预测结果计算 **RankIC / RankICIR** 并生成排行榜。

2. 提交目录结构

标准提交目录结构如下：

```
submission/
|-- inference.py
|-- train.py
|-- requirements.txt
`-- model/
    |-- model.joblib
    `-- meta.json
```

其中：

- `inference.py`
  推理入口。必须实现规定的 `InferenceModel` 类，并保证整个推理流程能够运行。
- `train.py`
  用于从公开的 2016–2022 年数据完整复现 `model/` 中提交的模型。
- `requirements.txt`

必须列出模型运行所需要的全部 Python 依赖，并声明相应版本。

- `model/`
  保存训练后的模型产物以及所有从公开训练数据中学习得到的参数。

提交包必须满足以下规则：

- **必须自包含。**
  `inference.py` 依赖的任何自定义模块均必须一并放入 `submission/` 目录。
  可以使用包内相对导入，例如：

```
from .helper import ...
```

- `model/` 中的文件数量和格式不限。
  可以使用包括但不限于：
  XGBoost、LightGBM、CatBoost、Random Forest、SVM、Ensemble 或深度学习模型权重。
  唯一要求是 `inference.py` 能够正常加载这些文件并完成推理。
- **所有从公开数据中学习得到的参数都必须固化到 `model/` 中。**
  例如：
  - 模型权重；
  - StandardScaler；
  - PCA；
  - 特征选择器；
  - Ensemble 权重；
  - 缺失值填充统计量；
  - 其他通过训练数据得到的参数。
- **`train.py` 必交。**
  必须能够从公开的 **2016–2022 年**数据完整复现提交的 `model/`。
  组织方会进行代码审查与抽样复跑。
  如果训练结果与提交模型不符，则比赛成绩无效。
- **`requirements.txt` 必交。**
  必须完整列出模型运行所需要的依赖并锁定版本。
  评测环境将根据该文件预装依赖。

3. inference.py

`inference.py` 必须实现如下类。

函数名称与类型固定，并与 QD 回测框架 `skd-backtest` 保持一致：

```
class InferenceModel:
    def __init__(self, model_dir: str):
```

```
        # Called once at the beginning.
        # Load models and all learned parameters here.
        ...

    def predict(
        self,
        as_of_date: str,
        data: dict[str, pd.DataFrame]
    ) -> pd.DataFrame:
        # Called once for every signal date.
        # Return scores for the full stock universe.
        ...
```

具体调用方式如下：

- `InferenceModel(model_dir="./submission/model/")`
  平台在评测开始时**只调用一次**。
  模型文件及其他固定参数应当在这里完成加载。
- `predict(as_of_date, data)`
  在此之后，平台会在**每个信号日调用一次**该函数。
- 平台在每次调用前会检查：
  - `as_of_date` 必须为 `str`；
  - `data` 必须为 `dict`；
  - `data` 的键必须为 `str`；
  - `data` 的值必须为 `pd.DataFrame`；
  - 返回值必须为 `pd.DataFrame`。

  类型不符合要求时，平台将直接抛出 `TypeCheckError`。
- `as_of_date` 是格式为

  `YYYY-MM-DD`

  的字符串，例如：

  ```
  "2022-01-04"
  ```

4. predict 的输入 data

`predict` 接收到的 `data` 是一个 Python `dict`。

其中包含三张**长表**，数据范围为截至当前交易日、最多 `lookback` 个交易日的研究历史。

键名保留源数据表的原始名称。

Key	内容	列
"MarketData"	行情	日期、代码、名称、open, high, low, close, volume, amount, is_suspend
"Factor33_winsor"	33 因子	日期、代码、名称以及 33 个因子列
"Barra_factor"	10 个风格因子	日期、代码、名称以及 10 个风格因子列

当日股票池 输入中**没有** universe 键。

当日合法股票池定义为：

当日 Barra_factor 表中出现的股票代码集合。

Barra_factor 表本身就是当日真实沪深 300 成分股。

推荐取法：

```
t = int(as_of_date.replace("-", ""))

barra = data["Barra_factor"]

universe = list(
    barra.loc[
        barra[" 日期"] == t,
        " 代码"
    ]
)
```

日期格式 **注意：输入日期格式存在不对称。**

传入的 as_of_date 是

"YYYY-MM-DD"

格式的字符串，

但三张数据表中的 **日期**列是整数格式：

YYYYMMDD.

因此在与表内日期比较前，需要进行如下转换：

```
t = int(as_of_date.replace("-", ""))
```

未来数据 平台保证三张输入表中的最大日期恰好等于 as_of_date。

输入数据中不会包含 as_of_date 之后的未来数据。

缺失值与预处理 平台会将原始数据直接传入模型。

其中：

- NaN 保持不变；
- 平台不负责补值；
- 平台不负责缩尾；
- 平台不进行标准化；
- 平台不进行其他特征变换。

因此，所有预处理操作均由参赛者自行完成。

Look-back Window 历史 lookback 窗口由参赛者自行决定。

如果模型需要若干天历史数据，应当自行从传入的长表中向前截取。

平台会根据比赛配置提供足够的历史数据。

股票代码与价格 股票代码包含市场前缀，例如：

```
SH600000
SZ000001
```

价格字段采用**后复权价格**。

5. predict 的输出

predict 必须返回一个 pd.DataFrame。

返回结果必须**恰好包含三列**：

$$\texttt{date}, \texttt{code}, \texttt{score}$$

例如：

```
date,code,score
2023-01-04,SH600519,0.8231
2023-01-04,SZ000858,0.7175
```

以下为**硬性规则**。

违反任意一条，评测平台将会报错，该次提交失败。

1. date 必须严格等于 as_of_date。
   并且必须是

`"YYYY-MM-DD"`

格式的字符串，不能输出整数日期。

2. `code` 必须覆盖当日合法股票池中的**每一只股票**，且每只股票**只能出现一次**。
   不允许：
   - 缺失股票；
   - 重复股票；
   - 混入合法股票池之外的股票。
3. `score` 必须是**有限实数**。
   禁止出现：
   - `NaN`；
   - `Inf`；
   - `-Inf`。
4. **score 越高表示模型越看好该股票。**
   `score` 不要求具有收益率量纲。
   平台只使用股票之间的相对排序，并会进行截面归一化。
5. 如果停牌股票 is_suspend = 1, 仍然属于当日合法股票池，则模型**仍然必须为其输出 score**，不允许省略。

平台会自动按照 code 对结果进行排序。参赛者无需自行排序。

6. 评测口径

比赛统一使用以下评测规则。

持有周期 持有周期为

$$H = 1$$

个交易日。

截面股票池为当日沪深 300 成分股，具体以当日 `Barra_factor` 数据为准。

评测标签 原始 O2O 收益定义为：

$$\text{future_return}(t) = \frac{\text{Open}(t+2)}{\text{Open}(t+1)} - 1.$$

RankIC 每个信号日计算：

$$\text{RankIC}(t) = \text{Spearman}\left(\text{score}(t), \text{future_return}(t)\right).$$

全评测期汇总以下指标：

- Mean RankIC；
- RankIC Standard Deviation；
- RankICIR；
- 正 IC 占比。

Public / Private Leaderboard Public Leaderboard 与 Private Leaderboard 分别在不同的隐藏时间区间上进行评测。

Private Leaderboard 作为最终排名依据。

7. 常见问题

如果提交报错，请逐项检查：

□ 目录结构为
`submission/inference.py` + `submission/train.py` + `submission/requirements.txt` + `submission/model/`。

□ `predict(self, as_of_date: str, data: dict[str, pd.DataFrame]) -> pd.DataFrame` 的签名与类型与本指南完全一致。

□ `data` 使用原始表键名：
`MarketData`、`Factor33_winsor`、`Barra_factor`。
合法股票池从当日 `Barra_factor` 中获得。

□ 已正确处理日期格式不对称：

- `as_of_date` 为字符串；
- 表内日期为整数；
- 输出 `date` 使用 `"YYYY-MM-DD"` 字符串。

□ 所有依赖模块都位于 `submission/` 内。
可以使用包内相对导入：

```
from .helper import ...
```

不依赖提交包之外的模块。

□ 所有通过公开训练数据学习得到的参数均已经固化到 `model/` 中。
`predict` 内：

- 不重新 `fit`；
- 不读取包外数据。

□ `train.py` 能够从公开数据完整复现 `model/`。
`requirements.txt` 已经完整列出全部依赖并锁定版本。

□ 输出严格为
`date`, `code`, `score`
三列。
并满足：

- 覆盖当日完整合法股票池；
- 无重复；
- 无 `NaN`；
- 无 `Inf`；
- 停牌股票仍然给分。

□ 已固定随机种子，保证同一提交在相同数据上结果可复现。

□ 已在本地使用样例数据运行完整的一段信号日，并确认不存在格式错误。

8. 运行环境

评测环境使用：

Python 3.11

Python 依赖以参赛者提交的 `requirements.txt` 为准。

评测环境会根据该文件预装相应依赖。

评测期间禁止联网。

`predict` 不得读取：

- `model/` 之外的本地数据；
- 平台传入 `data` 之外的数据；
- 任何远程数据或远程模型。

9. 代码审查与合规（反作弊）

为保证 Public Leaderboard 和 Private Leaderboard 的公平性与纯净性，组织方会对提交的

- `inference.py`；
- `train.py`；
- 以及提交包中的其他代码

进行抽查，以下行为一经发现，**取消评测资格。**

1. 触碰评测数据 / 未来信息 禁止：

- 在 `predict` 内使用 `as_of_date` 之后的任何数据；
- 跨信号日缓存、拼接或通过其他方式构造未来信息；

- 尝试探测、拟合或硬编码 Public / Private Leaderboard 的：
  - 时间区间；
  - 样本范围；
  - 标签；
  - 股票池。
- 使用评测期数据重新训练模型；
- 在评测阶段重新执行 `fit`；
- 使用评测数据在线更新模型参数。

2. 读取包外 / 外部资源 禁止访问 `submission/` 目录以及平台传入 `data` 之外的：

- 文件；
- 路径；
- 环境变量；
- 外部数据源。

允许使用包内相对导入，例如：

```
from .helper import ...
```

前提是对应自定义模块位于提交目录内部。

任何联网行为均被禁止。

禁止通过 `open()` 读写提交包之外的文件。

同样禁止将结果写入文件，并供后续信号日调用读取，从而形成跨调用共享状态。

3. 危险 / 规避审查的代码 禁止使用动态执行或代码混淆方法，包括但不限于：

- `eval`；
- `exec`；
- `compile`；
- `__import__`；
- 使用 `getattr` 等方法动态拼接并执行逻辑；
- Base64 解码后执行代码；
- Pickle 解码出的可执行逻辑；
- `marshal`；
- 字节码注入。

禁止使用子进程及系统调用，包括但不限于：

- `os.system`；
- `subprocess`；
- `os.popen`；
- `sys.exit`。

禁止通过反射等方式修改评测运行时，包括：

- 篡改平台评测代码；
- 修改 `sys.modules`；
- 修改平台全局状态。

4. 训练不可复现 以下情况视为训练不可复现：

- `train.py` 无法正常运行；
- 使用 `train.py` 复现得到的模型与提交的 `model/` 明显不符；
- 模型训练使用了 **2016–2022 年公开数据之外的数据源**。

由正常随机性造成的合理误差允许存在。